

```

# MEDSCAN — final build
# B1 short=Next/Confirm B2 short=Toggle/Switch Hold B1=Back Hold B2=Home
import time, json, os
from machine import Pin, SPI, SoftI2C
import ssd1306
from mfrc522 import MFRC522

# — PINS —————
i2c = SoftI2C(scl=Pin(22), sda=Pin(21), freq=400_000)
oled = ssd1306.SSD1306_I2C(128, 64, i2c)
spi = SPI(2, baudrate=2_500_000, polarity=0, phase=0,
        sck=Pin(18), mosi=Pin(23), miso=Pin(19))
spi.init()
rdr = MFRC522(spi=spi, gpioRst=4, gpioCs=5)
btn = [Pin(13, Pin.IN, Pin.PULL_UP), Pin(14, Pin.IN, Pin.PULL_UP)]
leds = [Pin(p, Pin.OUT) for p in (15, 16, 17, 25, 26)]
# leds[0-2]=dosage leds[3]=before_food leds[4]=after_food

# — STATES —————
WELCOME, MAIN_MENU, SET_SCAN, DOSAGE_ENTRY, FOOD_TIMING = 0, 1, 2, 3, 4
DATA_SAVED, LOAD_NEXT, READ_SCAN, DISPLAY_DATA, UNKNOWN_TAG = 5, 6, 7, 8, 9

# — GLOBALS —————
st = WELCOME
mode = 0 # 0=SET 1=READ
dos = [0, 0, 0] # dosage bits
fod = [0, 0] # food timing bits
di = 0 # dosage cursor
fi = 0 # food cursor
uid = "" # last scanned UID
done = False # dedup flag
save_mode = False # True = next scan saves
hist = [] # back-navigation stack
saved_at = 0 # timestamp of last save (ms) — used for non-blocking display

# — STORAGE —————
_DB = "ms.json"

def _db():
    try:
        with open(_DB) as f: return json.load(f)
    except: return {}

def db_save(u, v):
    d = _db(); d[u] = v
    with open(_DB, "w") as f: json.dump(d, f)

def db_get(u): return _db().get(u, "")

def db_clear():

```

```

try: os.remove(_DB)
except: pass

# — LEDs —————
def leds_off():
    for p in leds: p.value(0)

def leds_all_on():
    for p in leds: p.value(1)

def leds_show(d, f):
    for i in range(3): leds[i].value(d[i])
    leds[3].value(1 if f == [1, 0] else 0)
    leds[4].value(1 if f == [0, 1] else 0)

# — OLED helpers —————
def _hdr(title):
    oled.fill(0)
    oled.text(title, (128 - len(title) * 8) // 2, 2)
    oled.hline(0, 13, 128, 1)

def _footer(txt):
    oled.hline(0, 53, 128, 1)
    oled.text(txt, (128 - len(txt) * 8) // 2, 56)

def _show(): oled.show()

# — SCREENS —————
def scr_welcome():
    _hdr("MEDSCAN")
    oled.text("Hello!", 36, 25)
    oled.text("Welcome!", 24, 40)
    _show()

def scr_menu():
    _hdr("SELECT MODE")
    for i, (label, y) in enumerate([("SET MODE", 20), ("READ MODE", 36)]):
        if mode == i:
            oled.fill_rect(0, y, 128, 13, 1)
            oled.text("> " + label, 8, y + 2, 0)
        else:
            oled.text(" " + label, 8, y + 2, 1)
    _footer("B1:OK B2:Switch")
    _show()

def scr_scan(title, hint="Scan your tag"):
    _hdr(title)
    oled.text(hint, (128 - len(hint) * 8) // 2, 28)
    _footer("HoldB1=Back")
    _show()

```

```

def scr_dosage():
    _hdr("DOSAGE B2:0/1 B1:>")
    for i, x in enumerate((15, 50, 85)):
        oled.rect(x, 28, 20, 20, 1)
        oled.text(str(dos[i]), x + 6, 35)
    oled.text("^", 23 + di * 35, 50)
    oled.text("HoldB1=Back", 0, 56)
    _show()

def scr_food():
    _hdr("FOOD 10=Bfr 01=Aft")
    oled.text("B2:0/1 B1:Next", 0, 16)
    for i, x in enumerate((35, 70)):
        oled.rect(x, 30, 14, 14, 1)
        oled.text(str(fod[i]), x + 3, 34)
    oled.text("^", 41 + fi * 35, 46)
    oled.text("HoldB1=Back", 0, 56)
    _show()

def scr_saved():
    # — instant confirmation screen —————
    # shows dosage + food on same screen, NO blocking delay
    ft = "BEFORE" if fod == [1,0] else "AFTER" if fod == [0,1] else ""
    _hdr("*** SAVED! ***")
    oled.text("Dose: " + "".join(map(str, dos)), 4, 18)
    oled.text("Food: " + ft, 4, 30)
    _show()
    # — SAVED SCREEN AUTO-ADVANCE (non-blocking) —————
    # After 1.5 s on DATA_SAVED, auto-move to LOAD_NEXT — no sleep() anywhere
    SAVED_SHOW_MS = 9000    # ← change only this number

def scr_load_next():
    _hdr("Load next tag?")
    oled.text("B1: Yes, next", 8, 25)
    oled.text("B2: Main menu", 8, 40)
    _show()

def scr_data(data):
    d = [int(c) for c in data[:3]]
    f = [int(c) for c in data[3:5]]
    ft = "BEFORE" if f == [1,0] else "AFTER" if f == [0,1] else data[3:5]
    _hdr("TAG DATA")
    oled.text("Pills: " + data[:3], 0, 18)
    oled.text(ft + " food", 0, 32)
    _footer("B1:Scan B2:Menu")
    _show()
    return d, f

```

```

def scr_unknown():
    _hdr("!! UNKNOWN !!")
    oled.text("No data stored", 8, 28)
    _footer("B1:Scan B2:Menu")
    _show()

def scr_notice(msg):
    oled.fill(0)
    oled.text("< BACK", 36, 20)
    oled.text(msg, (128 - len(msg) * 8) // 2, 38)
    _show()
    time.sleep_ms(800)

# — NAVIGATION —————
_REDRAW = {
    MAIN_MENU: scr_menu,
    SET_SCAN: lambda: scr_scan("SET MODE"),
    READ_SCAN: lambda: scr_scan("READ MODE"),
    DOSAGE_ENTRY: scr_dosage,
    FOOD_TIMING: scr_food,
    LOAD_NEXT: scr_load_next,
}

def _push(s):
    hist.append(s)
    if len(hist) > 8: hist.pop(0)

def _goto(new_st, redraw=True):
    global st
    st = new_st
    if redraw and new_st in _REDRAW:
        _REDRAW[new_st]()

def _reset_entry():
    global dos, fod, di, fi
    dos = [0,0,0]; fod = [0,0]; di = 0; fi = 0
    leds_off()

def _reset_tag():
    global uid, done, save_mode
    uid = ""; done = False; save_mode = False

def go_home():
    hist.clear(); leds_off(); _reset_tag(); _reset_entry()
    _goto(MAIN_MENU)

def go_back():
    global di, fi, save_mode
    if not hist:
        scr_notice("Already home!")

```

```

scr_menu(); return
prev = hist.pop()
save_mode = False
if prev == DOSAGE_ENTRY and di > 0:
    di -= 1; _goto(DOSAGE_ENTRY); return
if prev == FOOD_TIMING:
    if fi > 0: fi -= 1; _goto(FOOD_TIMING); return
    else: di = 2; _goto(DOSAGE_ENTRY); return
if prev in (DISPLAY_DATA, UNKNOWN_TAG):
    leds_off(); _reset_tag(); _goto(READ_SCAN); return
if prev == WELCOME:
    _goto(MAIN_MENU); return
_reset_tag()
_goto(prev if prev in _REDRAW else MAIN_MENU)

```

— BUTTON ACTIONS —

```

def b1_short():
    global di, fi, st, save_mode

    if st == MAIN_MENU:
        _push(MAIN_MENU)
        if mode == 0: _reset_tag(); _reset_entry(); _goto(SET_SCAN)
        else: _reset_tag(); _goto(READ_SCAN)

    elif st == DOSAGE_ENTRY:
        di += 1
        if di >= 3: _push(DOSAGE_ENTRY); fi = 0; _goto(FOOD_TIMING)
        else: scr_dosage()

    elif st == FOOD_TIMING:
        fi += 1
        if fi >= 2:
            _push(FOOD_TIMING)
            _reset_tag()
            save_mode = True
            st = SET_SCAN
            scr_scan("Scan to SAVE")
        else: scr_food()

    elif st == DATA_SAVED:
        # B1 on saved screen → go straight to load-next
        hist.clear(); _push(MAIN_MENU); _goto(Load_NEXT)

    elif st == Load_NEXT:
        _push(Load_NEXT); _reset_tag(); _reset_entry(); _goto(SET_SCAN)

    elif st in (DISPLAY_DATA, UNKNOWN_TAG):
        leds_off(); _push(st); _reset_tag(); _goto(READ_SCAN)

def b2_short():

```

```

global mode
if st == MAIN_MENU:
    mode ^= 1; scr_menu()
elif st == DOSAGE_ENTRY:
    dos[di] ^= 1; scr_dosage()
elif st == FOOD_TIMING:
    fod[fi] ^= 1; scr_food()
elif st in (DATA_SAVED, LOAD_NEXT, DISPLAY_DATA, UNKNOWN_TAG):
    leds_off(); go_home()

# — RFID —————
def handle_rfid():
    global st, uid, done, save_mode, saved_at

    if st not in (SET_SCAN, READ_SCAN): return

    s, _ = rdr.request(rdr.REQIDL)
    if s != rdr.OK: done = False; return
    s, raw = rdr.anticoll()
    if s != rdr.OK: return

    tag = ("%02x%02x%02x%02x" % tuple(raw[:4])).upper()
    if tag == uid and done: return
    uid = tag; done = True
    print("TAG:", uid)

    if st == SET_SCAN:
        if save_mode:
            # — SAVE immediately, no blocking delay —————
            data = "".join(map(str, dos)) + "".join(map(str, fod))
            db_save(uid, data)
            print("SAVED:", uid, "->", data)
            leds_all_on()          # all LEDs on instantly as feedback
            save_mode = False
            saved_at = time.time() # record when saved (non-blocking)
            st = DATA_SAVED       # show saved screen immediately
            scr_saved()            # draw right away — zero delay
        else:
            print("FIRST SCAN - entering dosage")
            _push(SET_SCAN); _goto(DOSAGE_ENTRY)
    else:
        data = db_get(uid)
        _push(READ_SCAN)
        if len(data) == 5 and all(c in "01" for c in data):
            d, f = scr_data(data); leds_show(d, f); _goto(DISPLAY_DATA, False)
        else:
            scr_unknown(); leds_off(); _goto(UNKNOWN_TAG, False)

# — SAVED SCREEN AUTO-ADVANCE (non-blocking) —————
# After 1.5 s on DATA_SAVED, auto-move to LOAD_NEXT — no sleep() anywhere

```

```
SAVED_SHOW_MS = 1500
```

```
def check_saved_timeout():  
    if st == DATA_SAVED and time.ticks_diff(time.ticks_ms(), saved_at) >= SAVED_SHOW_MS:  
        leds_off()  
        hist.clear(); _push(MAIN_MENU); _goto(Load_Next)
```

```
# — BOOT —————  
db_clear(); leds_off(); scr_welcome(); time.sleep_ms(2000)  
hist.clear(); _goto(MAIN_MENU)
```

```
# — MAIN LOOP
```

```
# bs[i] = [last_val, down_at, held_fired, last_short_t]  
bs = [[1, 0, False, 0], [1, 0, False, 0]]  
HOLD = 1000  
DEB = 250
```

```
while True:  
    now = time.ticks_ms()  
  
    for i, b in enumerate(btn):  
        v = b.value()  
        lst = bs[i]  
        pv = lst[0]  
  
        if v == 0 and pv == 1:      # pressed  
            lst[1] = now; lst[2] = False  
  
        elif v == 0 and pv == 0:    # held  
            if not lst[2] and time.ticks_diff(now, lst[1]) >= HOLD:  
                lst[2] = True  
                (go_back if i == 0 else go_home)()  
  
        elif v == 1 and pv == 0:    # released  
            if (not lst[2]  
                and time.ticks_diff(now, lst[1]) < HOLD  
                and time.ticks_diff(now, lst[3]) > DEB):  
                lst[3] = now  
                (b1_short if i == 0 else b2_short)()  
  
        lst[0] = v  
  
    check_saved_timeout() # auto-advance DATA_SAVED → Load_Next  
    handle_rfid()  
    time.sleep_ms(10)
```